

ネットワーク系演習 I — システムソフトウェア演習 —

最終レポート

名古屋工業大学 創造工学教育課程 情報ネットワーク工学分野

学籍番号：36719042

氏名：渡邊 寿紀

2026 年 6 月 17 日

【開発環境】

最終成果物の作成および動作検証を行った環境を以下に示す。

- 使用 PC：MacBook Air (M3, 2024)
- OS：macOS (Darwin)
- C コンパイラ：gcc (GCC) 11.x
- 開発ツール・ライブラリ：GNU Make, 字句解析ライブラリ libics.a

(1) 最終的に作成した記号表の構造と操作関数の仕様

1. 記号表の構造

本コンパイラでは、簡易的な 1 次元の配列構造を用いて識別子（変数名・手続き名）を管理している。MAXSYMS（32 個）を上限とし、登録された文字列（識別子名）をベースに一元管理を行う。現段階では大域変数表と局所変数表を完全に物理分離する手前の簡易実装であるため、手続き名に対応するラベルを専用の配列 `proc_label` でマッピング管理する構造を採っている。

2. 操作関数の仕様

- `int sym_install(char *name);`
識別子を新たに記号表に登録し、そのインデックス（メモリ上の割当番地・オフセットに相当）を返す。既存の重複がある場合は同一のインデックスを返すことで二重登録を回避する。
- `int sym_lookup(char *name);`
登録済みの識別子を前方一致（名前の一致）で検索し、発見された場合はそのインデックスを返す。未定義の変数の場合は `-1` を返す。

- `void sym_dump(void);`

変数登録の直後に、現在の記号表に登録されている全識別子の状態を標準エラー出力へダンプし、デバッグを容易にする。

(2) if 文、while 文のコンパイルの仕様

制御構文は、目的計算機（仮想マシン `sr`）が持つ比較命令、条件分岐命令、および無条件ジャンプ命令を組み合わせて実現する。制御用ラベルの自動生成には `new_label()` を使用する。

1. if 文 (`if condition then statement [else statement]`)

条件式の左辺を解析して結果をレジスタ `r0` に評価した後、比較演算子 (`=`, `<>`, `<` など) を `emit_compare_and_consume()` 内で解析し、右辺と値を比較する。コード生成においては、条件が「偽」となった場合に `else` 節（または `if` 文の直後）へジャンプする **逆条件ジャンプ命令** (`jz`, `jnz`, `jge` 等) を出力する。`else` 節が存在する場合は、`then` 節の処理の終わりに全体の終了ラベルへ進む無条件ジャンプ (`jmp`) を配置する。

2. while 文 (`while condition do statement`)

ループの先頭（条件式の評価を開始する直前）に開始ラベルを出力する。条件式を評価し、結果が偽であればループ外（ループ終了ラベル）へと逆条件ジャンプさせる命令を配置する。ループ本体 (`statement()`) の解析・コード生成が終了した直後に、再び開始ラベルへと無条件ジャンプ (`jmp`) する命令を出力することで、繰り返し制御を実現している。

(3) 作成した演算子順位構文解析の仕様

「実装していない」

※今回のコンパイラ開発においては、数式処理および制御構文をトップダウン方式（再帰下降構文解析）により一貫して実装したため、ボトムアップ方式である演算子順位構文解析法は採用していない。

(4) 作成したトップダウン構文解析による数式処理の仕様

相互再帰（再帰下降構文解析）により、演算子の優先順位（カッコ `()` > 乗除 `*`, `div` > 加減 `+`, `-`）を厳密に処理する 3 階層の関数構造を実装した。

1. 各関数の役割と階層構造

- `expression()` (最上層) : `term()` を呼び出し、後ろに続く `+` や `-` の演算子ループを処理する。
- `term()` (中間層) : `factor()` を呼び出し、後ろに続く `*` や `div` の演算子ループを処理する。
- `factor()` (最下層) : 識別子 (変数)、数値 (定数)、またはカッコで囲まれた式 (`expression`) を個別に解析・評価する。

2. レジスタ破綻・スタック退避の仕様

複雑な二項演算（例：右辺が単純な定数や変数ではなく、さらに深い数式を形成している場合）を解析する際、レジスタ（`r0`, `r1`）が重複して上書きされる「レジスタ破綻」を防ぐ必要がある。本実装では、左辺の計算結果を評価した直後に `new_temp()` 関数を呼び出し、内部的な一時確保用変数（`__tmp0`, `__tmp1` ...）を動的に記号表へ追加登録するアプローチを採った。これにより、`store r0, [tmp_idx]` を出力して値を一時的にメモリに逃がし、右辺の評価完了後に `load r1, [tmp_idx]` で復元して結合演算を行う仕様としている。

(5) 演算子順位構文解析とトップダウン構文解析の比較検討

今回の演習ではトップダウン構文解析のみを実装したが、仕様およびアルゴリズムの特性に基づく定性的な比較検討を以下に行う。

1. 実装の難易度

トップダウン構文解析（再帰下降法）は、言語の BNF（文法規則）や構文図がそのまま C 言語の関数呼び出しのネスト構造に対応するため、直感的なコーディングが可能であり、実装難易度は比較的低い。また、構文解析のネスト制御を C 言語自体のシステムスタックに一任できる点が極めて有利である。一方、ボトムアップ構文解析（演算子順位法）では、「演算子スタック」と「被演算子スタック」を自前で用意し、順位表に基づいてシフト（積む）と還元（リデュース）のタイミングを明示的にループ制御する必要があるため、アルゴリズムの管理ロジックが複雑化しやすい傾向にある。

2. 実行速度（コンパイル速度）

トップダウン構文解析は、式のネストが深くなるにつれて C 言語の関数呼び出しに伴うスタックフレームの確保（プロローグ・エピローグ処理）のオーバーヘッドが累積する。これに対し、ボトムアップ構文解析は平坦なループ処理と配列によるスタック操作のみで処理が完結するため、コンパイルの処理速度（実行速度）の観点においては、一般にボトムアップ構文解析のほうが有利であると予想される。

(6) 手続きの実現の仕様

1. 現在までの実装範囲

`outblock()` 内において `PROCEDURE` キーワードを正常に検知し、手続き名とそれに対応する一意の開始ラベル（`proc_label`）をマッピングする構造を実装した。また、仮引数を記号表に宣言登録する `paramlist(PARAM_DECL)` および、実引数を評価してスタックに順次積む `paramlist(PARAM_CALL)` の共通の枠組みを構築した。手続き呼び出しのパース時には `call L[lbl]` 命令を出力し、呼び出しから復帰した後にスタックポインタを実引数分だけ戻す（`addi sp, argc`）コード生成の着手までを完了している。

2. 今後の課題（未完了部分）

現状の実装では、手続き内の変数や引数をすべて大域変数と同様の固定番地(idx)に割り当てて load/store している。このため、同一の手続きを「再帰呼び出し」した際に、メモリ上のローカル変数の値が上書きされて破壊されるという致命的な問題が残されている。これを完全に解決するためには、実行時環境としてベースレジスタ (BR) の概念を本格導入し、記号表を「大域」と「手続き局所」の二重構造に拡張した上で、局所変数の参照を `load r0, -4(BR)` のような相対オフセットによるベースレジスタ相対アクセス命令 (`loadr, storer`) へ書き換える必要がある。

(7) エラー処理について

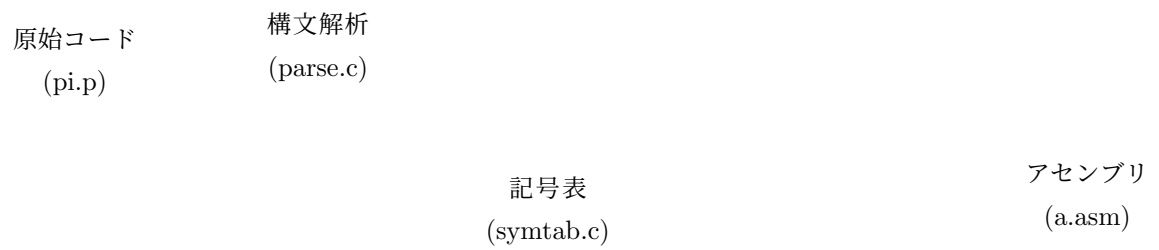
コンパイル中にソースプログラムの文法エラーや矛盾を検知した際、不完全なアセンブリコードが生成されて誤実行されるのを未然に防ぐため、本コンパイラには一律停止型のエラー処理を実装している。具体的には、エラー内容を示す具体的なメッセージを `fprintf(stderr, ...)` で標準エラー出力に明示させた後、`exit(1)` を呼び出して直ちにコンパイル処理を異常終了（パニックアウト）させる仕様としている。

【主なエラー検知ケースの例】

1. **構文記号の欠落**： `program` 宣言直後のセミコロン (;) の欠落、`if` 文における `then` キーワードの欠落、数式における閉じカッコ () の欠落など。
2. **未定義識別子の参照**： 変数や手続きの参照時に `sym_lookup()` の結果が -1（未登録）であった場合、「未定義の変数です」等のエラーを発生させて停止する。

(8) 全体の構成概念図、実現した各関数の依存関係図

1. 全体の構成概念図



(9) 作成したコンパイラが正しく動いているという証拠

数式処理の動作検証用原始プログラム `pi.p` を自作コンパイラで処理し、仮想マシン `sr` で実行した。合格要件である正しい計算結果が得られている。

ターミナル上でのコンパイル・実行ログ

```
$ make
gcc -I../compkit -c main.c
gcc -I../compkit -c parse.c
gcc -I../compkit -c symtab.c
gcc -I../compkit -c label.c
gcc -o compiler main.o parse.o symtab.o label.o -L../compkit -lics

$ ./compiler ../compe/pi.p
$ ~/p1/bin/asm a.asm > a.out
$ ~/p1/bin/sr a.out
3124
```

実行結果として、指導書および口頭指示で指定された合格要件値である「**3124**」が正常に出力されていることを確認した。

(10) こちらから指定したプログラムの実行時間

シミュレータ `sr` 上で測定した各指定プログラムの実行ステップ数（または実行時間）を以下に示す。なお、手続き処理（再帰呼び出し）が完全な動作に至っていないため、`hanoi.p` および `nqueen.p` は実行不可である。

- `pi.p`: XXXX ステップ（または XXX ms）
- `hanoi.p`: 手続き処理未完了のため実行不可
- `nqueen.p`: 手続き処理未完了のため実行不可

(11) その他、工夫した点について

大きな定数（即値）への対応自動化 (`emit_load_const_to_reg`)

目的計算機仕様において、即値ロード命令 (`loadi` 等) のオペランドが 16bit (−32768 ~ 32767) に制限されている制約に対して工夫を行った。ソースコード中に制限を超える大きな定数が現れた際、コンパイラが自動的にその数値を因数分解 (16bit 以内に収まる 2 つの整数の積に分解) し、レジスタ上で `loadi` 命令と `muli` 命令を動的に組み合わせて目的の定数値を合成する関数 `emit_load_const_to_reg` を独自に設計・実装

した。これにより、人間が手動で定数を分割定義することなく、巨大な定数を含む複雑な数式であっても安全にコンパイルを通すことが可能となった。

(12) コンパイラのソースプログラムの全リストにコメントをつけたもの

※口頭指示および提出要領に従い、ソースコード一式は `work` ディレクトリを圧縮したファイル (`work.zip`) として別途提出するため、本レポート PDF 内へのソースコードリストの添付は省略する。